

HOW THE WHOLE APP CONNECTS

Six plates, drawn from the source rather than from memory: what talks to what across the network, how a URL becomes a guarded page, where every piece of state actually lives, and the three flows — a task, an hour, a family’s access code — that carry the product. One app ships two products from one bundle: a staff operations tool behind a login, and a public parent portal with no account at all.

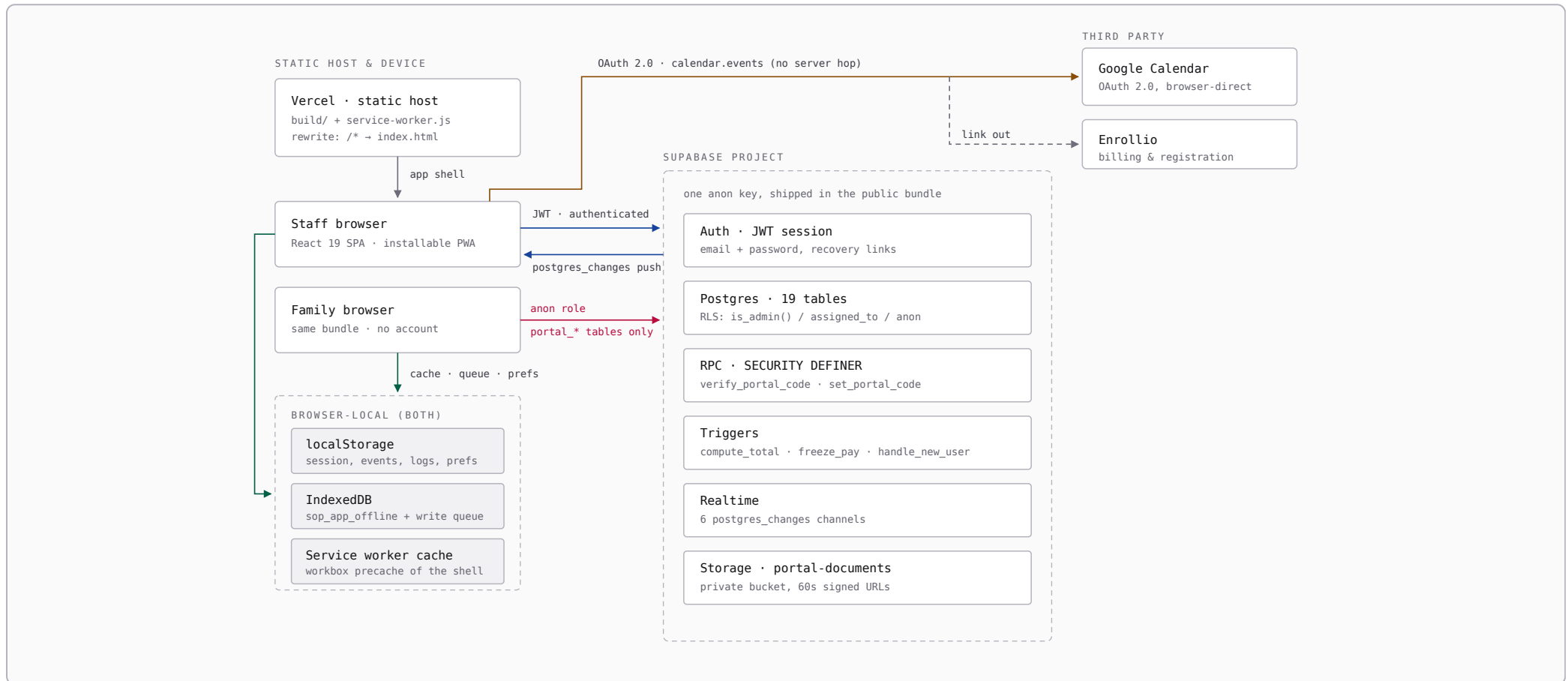
SOURCE `conquerdesigngroup/sop-app @ 9a86370` STACK `React 19 · TypeScript · Supabase · Vercel` SURFACE `18 staff pages · 8 portal pages · 19 tables`

 authenticated read/write (JWT)  public / anon surface  auth, gate or approval step  device-local storage

--- conditional or dormant path

ONE BUNDLE, ONE KEY, TWO AUDIENCES

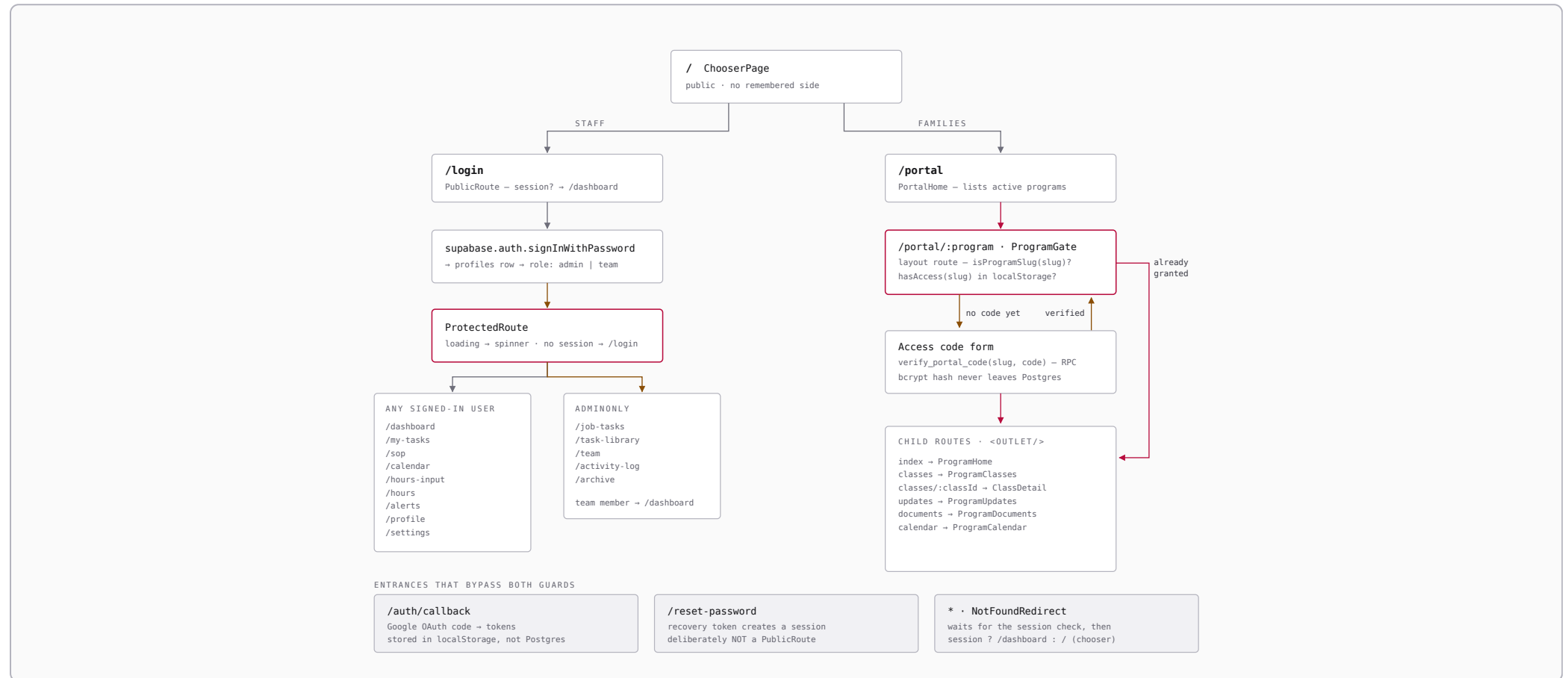
There is no backend of our own. Vercel serves a static React build; every request after that goes from the browser straight to Supabase using the anon key that ships inside the public bundle. That single fact sets the security model: what a signed-out visitor may see is decided entirely by row-level security in Postgres, not by anything the client does.



Read it left to right. Vercel is only a file server – it never sees app data. The two browsers run identical code and differ only in the Postgres role their requests carry, which is why the anon predicate on portal_* is the whole boundary between a family and the staff tables. Google Calendar is called from the browser with a token kept in localStorage; Enrollio is a plain outbound link.

THE FORK AT THE FRONT DOOR

/ is a chooser, not a login — and it is shown on every visit with no remembered side. Staff go left into a session-guarded app where a second guard splits admin from team. Families go right into pages that never authenticate at all; their gate is a bcrypt code checked by a database function, remembered per device.



Two guards, two mechanisms. The staff guard is client-side and advisory – it decides what renders, while RLS decides what is actually returned. The family gate is server-side: `verify_portal_code` is SECURITY DEFINER over a table with no grants, so a wrong guess learns nothing. Staff chrome is suppressed on portal paths by `showStaffChrome = isAuthenticated && !isPortalPath(pathname)`, which is what keeps an admin previewing the portal from seeing the staff header bleed through.

FIFTEEN LAYERS, ONE STRAIGHT LINE

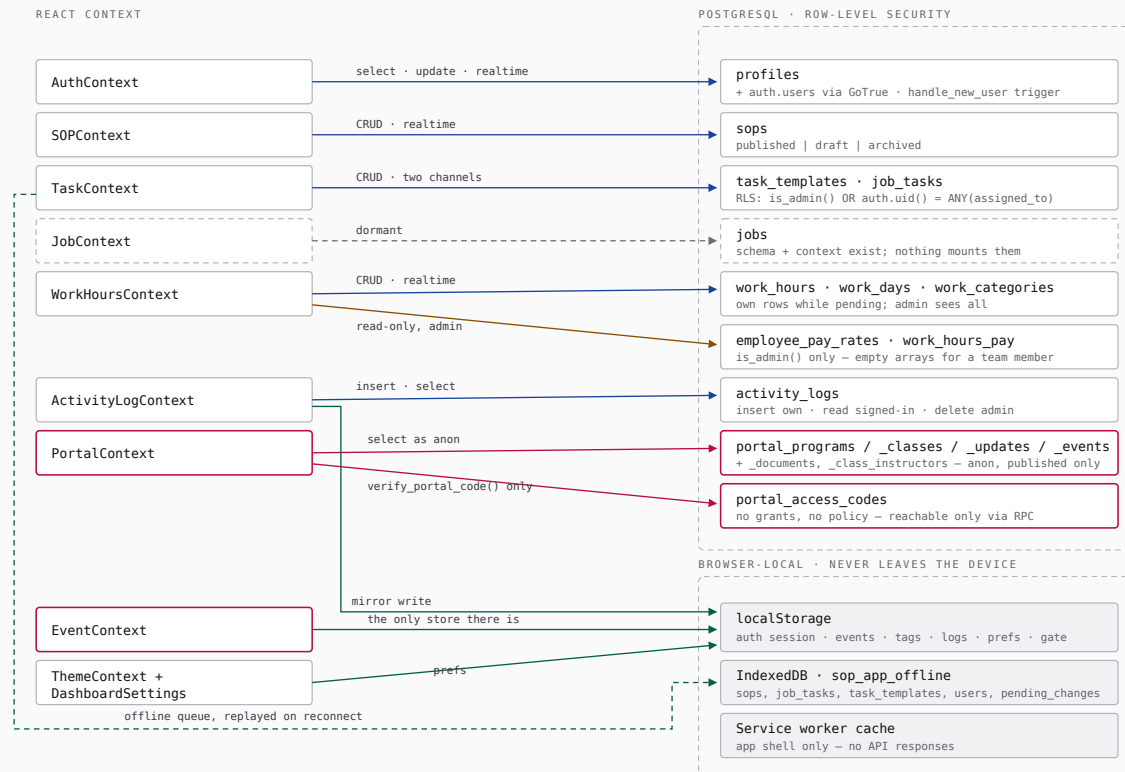
Every provider in `App.tsx` wraps the next — there are no siblings, so the tree is really a chain. That matters for two reasons: anything above Router mounts on *every* route including the public portal, and a context that fetches on mount without checking for a session will fire requests on a family’s device. Only `PortalProvider` is scoped to its routes.



Two rows to remember. `EventProvider` is the only data context with no table behind it — calendar events live in one browser and are invisible to everyone else. `JobProvider` and its jobs table are wired up but unreachable: the only component calling `useJob()` is `JobForm`, which no page imports.

EVERY CONTEXT, AND WHAT BACKS IT

Pages hold almost no data of their own — they read a context, and each context owns exactly one slice of storage. Trace any value on screen back through this plate and you land on either a Postgres table with an RLS predicate, or a key in the browser that no one else can see.

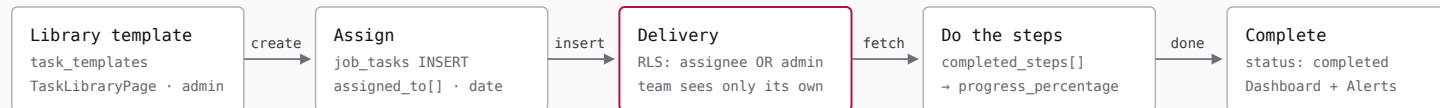


The right-hand column is the real API. Nine contexts, nine slices — near enough one-to-one, which is why almost every wire runs straight across. The exceptions carry the warnings: pay tables are a second, admin-gated read off the hours context; the portal access code is reachable only through a function, never a table read; and EventContext's only wire goes to the device.

THREE FLOWS THAT CARRY THE PRODUCT

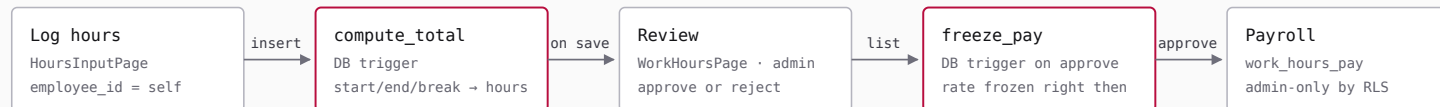
Everything else is scaffolding around these: a task moving from a library entry to a completed checklist, an hour moving from a phone in the field to frozen payroll, and a family moving from a link to a document. Each one crosses a boundary where the database, not the client, is the authority.

A · A TASK – FROM LIBRARY TO DONE



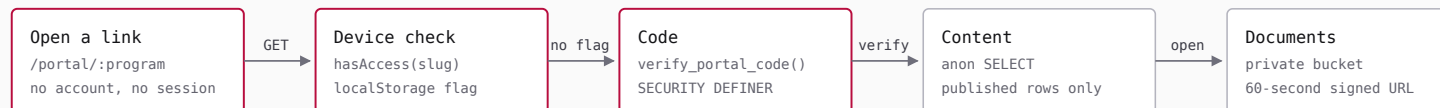
Archive instead of delete: status → archived, visible on /archive, restorable by an admin.
Every transition also writes an activity_logs row.

B · AN HOUR – FROM ENTRY TO PAYROLL



The rate is frozen at first approval, not read live – a later rate change never rewrites history.
No rate on file sets rate_missing rather than silently paying zero. Rejected entries return to pending and become editable again.

C · A FAMILY – FROM LINK TO DOCUMENT

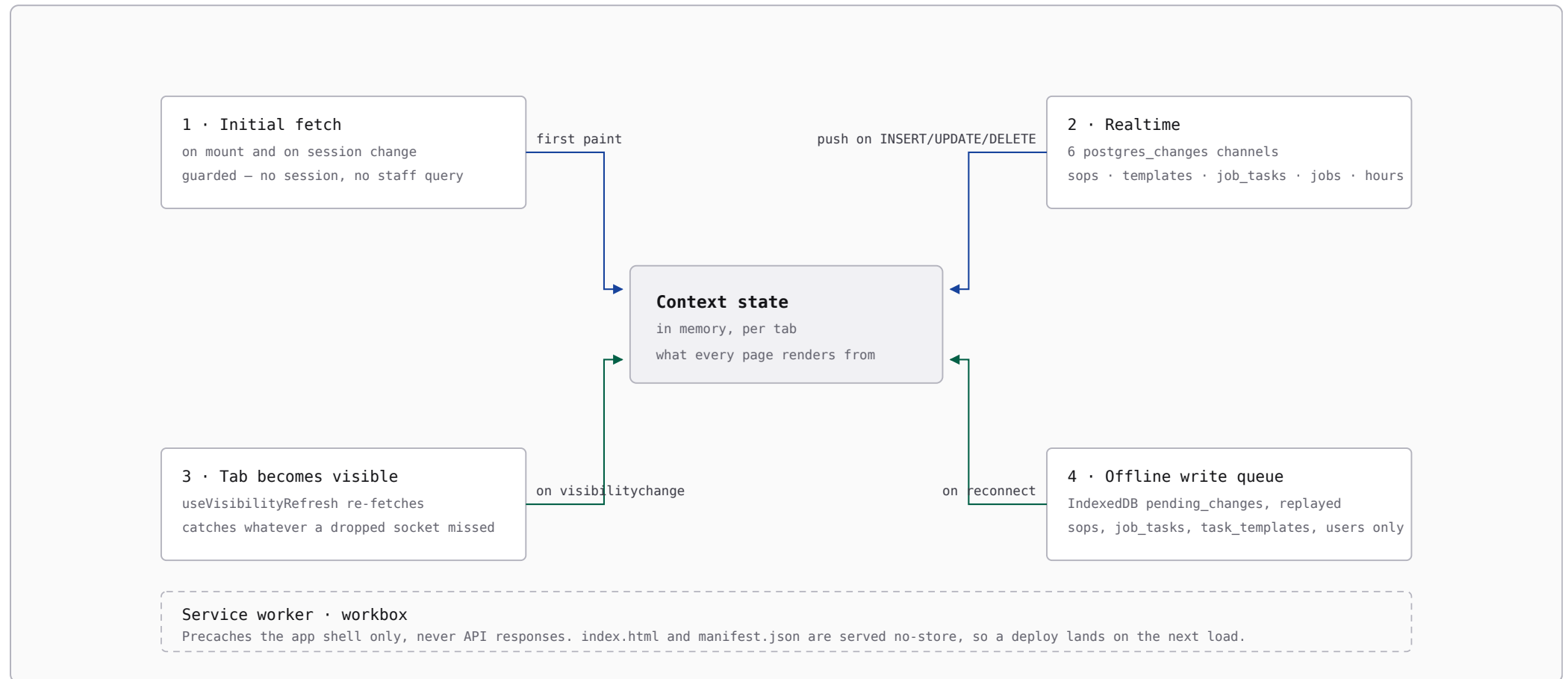


A soft gate by design: the hash lives in a table with no grants, but portal content itself is anon-readable, so the code stops casual browsing and nothing more. Nothing private belongs there.

Highlighted boxes are where control leaves the browser. RLS decides which tasks come back, two Postgres triggers own the arithmetic and the pay snapshot, and a SECURITY DEFINER function owns the access code. Nothing on these three paths can be talked out of its answer by the client.

FOUR WAYS STATE GETS CURRENT

The app is used on phones, in buildings, by people who lock the screen mid-task. So no single refresh path is trusted: a fetch on mount, a live socket, a re-fetch when the tab comes back, and a queue for writes made with no connection at all.



The queue is narrower than it looks. Only four stores replay on reconnect – hours, events, portal content and activity logs are not queued, so a write made offline on those screens is lost rather than deferred.

EVERY URL, END TO END

The same map as Plate 2, in the form you actually search: URL, the component behind it, who gets in, and what it reads.

ROUTE	COMPONENT	ACCESS	CONTEXTS	BACKING DATA
/	ChooserPage	public	Theme	– (static + Enrollio link)
/login	Login	public	Auth, Theme	auth.users, profiles
/auth/callback	AuthCallback	public	–	Google OAuth → localStorage
/reset-password	ResetPassword	public	–	auth.users (recovery token)
/dashboard	Dashboard	signed in	Auth, SOP, Task, Event, WorkHours, DashboardSettings	sops, job_tasks, work_days, localStorage
/my-tasks	MyTasksPage	signed in	Auth, Task, SOP, Toast	job_tasks (own rows), sops
/sop	SOPPage	signed in	Auth, SOP, Confirm	sops
/calendar	CalendarPage	signed in	Auth, Event, Task, WorkHours, Toast	job_tasks, work_days, localStorage events
/hours-input	HoursInputPage	signed in	Auth, WorkHours, Toast, Confirm	work_hours, work_categories
/hours	WorkHoursPage	signed in	Auth, WorkHours, Toast, Confirm	work_hours, work_days, pay tables (admin)
/alerts	AlertsPage	signed in	Auth, Task, Toast	job_tasks (derived, no table of its own)
/profile	ProfilePage	signed in	Auth, Toast	profiles
/settings	SettingsPage	signed in	Auth, SOP, Task, Theme, Toast	prefs + data-integrity checks across tables
/job-tasks	JobTasksPage	admin	Auth, Task, SOP, Toast, Confirm	job_tasks, task_templates
/task-library	TaskLibraryPage	admin	Task, Toast, Confirm	task_templates
/team	TeamManagementPage	admin	Auth, Toast, Confirm	profiles, employee_pay_rates
/activity-log	ActivityLogPage	admin	Auth, ActivityLog	activity_logs + localStorage mirror
/archive	ArchivePage	admin	SOP, Task	sops + job_tasks where status = archived
/portal	PortalHome	public	Portal	portal_programs
/portal/:program	ProgramGate → ProgramHome	code	Portal	portal_programs, verify_portal_code()
/portal/:program/classes	ProgramClasses	code	Portal	portal_classes, portal_class_instructors
/portal/:program/classes/:id	ClassDetail	code	Portal	portal_classes, portal_updates
/portal/:program/updates	ProgramUpdates	code	Portal	portal_updates (published only)
/portal/:program/documents	ProgramDocuments	code	Portal	portal_documents + signed storage URLs
/portal/:program/calendar	ProgramCalendar	code	Portal	portal_events
*	NotFoundRedirect	public	Auth	–

WHAT THE DIAGRAMS WON'T WARN YOU ABOUT

Seven things that are true of this codebase today and are easy to get wrong when changing it.

CALENDAR EVENTS NEVER LEAVE THE DEVICE

EventContext reads and writes three localStorage keys and has no Supabase call at all — no events table exists. Two people looking at the same calendar see different events, and clearing site data deletes them. Tasks on that same calendar *are* shared, which makes the split easy to miss.

`src/contexts/EventContext.tsx`

PAY DATA IS GATED IN THE DATABASE, NOT THE UI

employee_pay_rates and work_hours_pay are RLS-gated on is_admin(), so for a team member the context arrays are simply empty. There is no client-side filter to preserve — and equally, none to rely on if you ever loosen those policies.

`src/contexts/WorkHoursContext.tsx` · `migration v7`

ACTIVITY LOGS ARE WRITTEN TWICE

logActivity() inserts into activity_logs and mirrors to localStorage, and the log page reads both. The mirror is per-device and uncapped, so an admin's history looks different on their laptop than on their phone.

`src/utils/activityLogger.ts` · `src/contexts/ActivityLogContext.tsx`

OFFLINE COVERS FOUR STORES, NOT THE APP

The IndexedDB queue replays sops, job_tasks, task_templates and users. Hours, calendar events, portal content and logs are not queued — on those screens an offline write fails rather than waiting for reconnection.

`src/lib/indexedDB.ts` · `src/hooks/useOfflineSync.ts`

STAFF CONTEXTS MOUNT ON PORTAL ROUTES

DataProvider sits above Router, so every staff context mounts for a signed-out family too. WorkHoursContext short-circuits its fetch and its channel when there is no session, which is what got a portal load down to three requests. A new context that fetches unguarded puts those requests back.

`src/App.tsx` · `src/contexts/PortalContext.tsx` (header)

THE PORTAL GATE IS SOFT, ON PURPOSE

Portal content is readable by the anon role because a portal with no login has to be. The bcrypt code stops casual browsing; it cannot stop anyone holding the anon key, which ships in the public bundle. Nothing private belongs in portal_* content.

`migration v9 header` · `src/pages/portal/ProgramGate.tsx`

JOB S ARE WIRED BUT UNREACHABLE

JobProvider mounts, subscribes to a realtime channel and queries the jobs table on every staff load, but the only consumer of useJob() is JobForm, which no page imports. work_schedule_templates has no client code at all. Both are cost with no surface.

`src/contexts/JobContext.tsx` · `src/components/JobForm.tsx`

THEME TOKENS ARE CSS VARIABLES, NOT HEX

theme.colors.bg.* resolves to var(--c-bg-primary, ...). Concatenating an alpha suffix onto one, or passing one as an SVG presentation attribute, silently produces an invalid value — the element renders with no colour rather than throwing.

`src/theme.ts` · `CLAUDE.md`